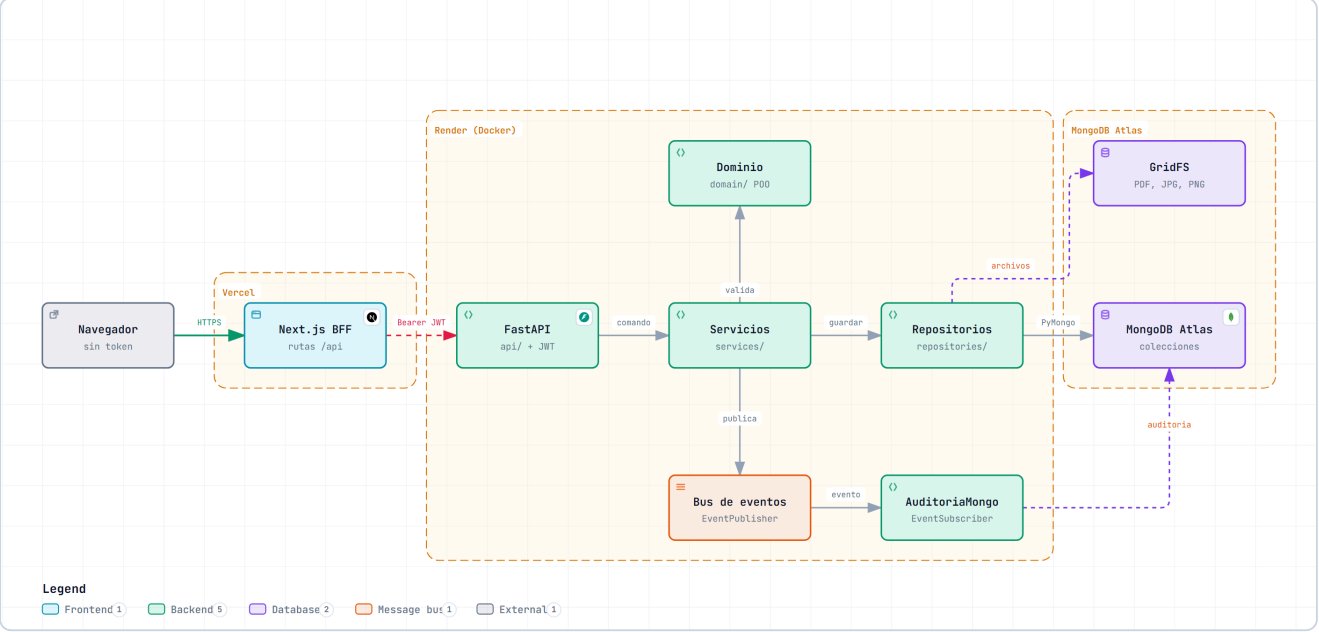
Guía técnica de sustentación

Gestión Documental de cuentas de cobro. Python 3.14, FastAPI, MongoDB Atlas, Next.js.

Repositorio: https://github.com/Juanxo17/gestion-documental · Plataforma: https://gestion-documental-livid.vercel.app

1. Arquitectura por capas



Capa	Carpeta	Responsabilidad	Depende de
Presentación	`frontend/`	Pantallas Next.js. Patrón BFF: el JWT vive en una cookie httpOnly y el navegador nunca lo ve	API (vía `/api/backend`)
API	`api/`	HTTP: valida la entrada, autentica, autoriza por permiso y traduce errores a códigos HTTP	Servicios
Servicios	`services/`	Orquesta un caso de uso: comando, validaciones con repositorios, dominio, guardar, publicar	Dominio, repositorios, publicador
Dominio	`domain/`	Reglas del negocio puras. No importa nada de FastAPI ni de PyMongo	Nada
Eventos	`events/`	Comandos y eventos inmutables (Pydantic), `EventPublisher`, `EventSubscriber`, suscriptores	Dominio
Persistencia	`repositories/`	Interfaces por agregado; implementación MongoDB (incluido GridFS) y en memoria	Dominio

Reglas de dependencia: las flechas apuntan hacia el dominio; el dominio no conoce a nadie; el servicio depende de interfaces, no de MongoDB; el publicador no conoce a sus suscriptores.

Todo lo que se inyecta se arma en `api/dependencias.py`: un dataclass `Repositorios` con todos los repositorios, y a partir de él cada servicio y el bus. Los tests reemplazan solo ese punto con repositorios en memoria.

2. Flujo comando, dominio, repositorio, evento, suscriptores

Ejemplo: aprobar una cuenta (`POST /cuentas/{radicado}/estado`).

1. **API** (`api/rutas_cuentas.py`). La dependencia `requiere_permiso("cuentas.cambiar_estado")` lee el JWT de la cookie (reenviado por el BFF) y verifica el permiso del rol. Si falta, responde 403. El `id_usuario` sale del token, nunca del cuerpo de la petición.
2. **Comando** (`events/comandos.py`). Se construye `CambiarEstadoCuenta` , inmutable (`frozen=True` , `extra="forbid"`).
3. **Servicio** (`services/servicio_cuentas.py` , `cambiar_estado`). Obtiene la cuenta del repositorio.
4. **Dominio** (`domain/cuenta_cobro.py` , `CuentaCobro.cambiar_estado`). Valida la transición contra la tabla `TRANSICIONES` . Si es inválida, lanza `TransicionInvalidaError` , que la API traduce a 409.
5. **Repositorio** (`repositories/mongo.py`). `guardar` hace un upsert por `numero_radicado` , con documentos e historial embebidos.
6. **Evento**. Solo si guardar tuvo éxito, el servicio publica `EstadoCuentaCambiado` (o `CuentaPagada` / `CuentaAnulada` , que heredan de él).
7. **Suscriptores** (`events/publicador.py` , `BusEventosEnMemoria.publicar`). Recorre el MRO del evento y entrega a `AuditoriaMongo` (suscrito a `EventoDominio`) y al `NotificadorSupervisor` (suscrito a `EstadoCuentaCambiado`). Si un suscriptor falla, se registra con `logging` y los demás siguen.

3. Decisiones y alternativas descartadas

Decisión	Por qué	Alternativa descartada
Un repositorio por agregado, sin repositorio genérico	Cada agregado se consulta distinto; interfaces pequeñas se implementan dos veces (Mongo y memoria) y se prueban con la misma batería	`Repositorio[T]` con CRUD genérico: métodos sin uso y filtros sueltos
Documentos e historial embebidos en la cuenta	Siempre se leen con la cuenta; una sola lectura	Colecciones separadas con referencias, como en SQL
Referencias por `_id` entre agregados y llaves naturales como índices únicos	Conserva la decisión del diseño SQL: llave sustituta estable y natural única	Usar la llave natural como referencia
`CHECK` del SQL como validación del dominio	Se prueban sin red y viven en un solo lugar	Validación `$jsonSchema` en MongoDB
`CuentaCobro` por composición, con estado derivado del historial	Una sola fuente de verdad; no se puede saltar `cambiar_estado`	`estado` como atributo público editable
Jerarquía de documentos de máximo 3 niveles	Evita la fragilidad de jerarquías profundas	`DocumentoPDFFirmado(DocumentoPDF)`
Eventos y comandos con Pydantic `frozen`	Un hecho publicado no cambia; datos validados al crearse	Diccionarios sueltos
Bus síncrono en memoria tras una interfaz `EventPublisher`	Suficiente para un proceso; se cambia por RabbitMQ o Kafka sin tocar servicios	Llamar directamente a auditoría y notificación desde el servicio
Suscriptores tipados con un `Protocol`	Duck typing verificado por mypy	Exigir `isinstance(EventSubscriber)`
JWT en cookie httpOnly detrás de un BFF	El token nunca toca el JavaScript del navegador	Guardar el token en `localStorage`
Bcrypt con `pwdlib` y reloj inyectable	Hash irreversible con sal; el bloqueo de 15 min se prueba sin esperar	Contraseñas en texto o hash sin sal
Tipo de archivo detectado por contenido	Un `.exe` renombrado a `.pdf` se rechaza	Confiar en la extensión
Descarga que recalcula el sha256 antes de enviar	Nunca se entrega un archivo alterado (409)	Enviar sin verificar

4. Matriz de trazabilidad completa

Las rutas de código son relativas a `src/gestion_documental/`.

#	Tema	Dónde está	Cómo demostrarlo en vivo
1	Abstracción	`domain/documentos.py`, `Documento` (ABC)	`uv run python demo.py` (sección 1)
2	Abstracción y NoSQL	`repositories/base.py`, `mongo.py`, `memoria.py`, `mapeo.py`	`uv run python demo.py` y luego `uv run python demo.py --mongo`
3	Base de datos	`repositories/mapeo.py`, `repositories/indices.py`, `scripts/seed.py`	`uv run python scripts/seed.py`; `uv run pytest -m integracion tests/test_indices.py`
4	Arquitectura basada en eventos	`services/servicio_cuentas.py`, `events/`	`demo.py` sección 4; pantalla Auditoría
5	EventPublisher y EventSubscriber	`events/publicador.py`	`uv run pytest tests/test_bus_eventos.py`
6	Manejo polimórfico de eventos	`BusEventosEnMemoria.publicar` (recorre el MRO)	`uv run pytest tests/test_servicios_api_completa.py -k sin_cambiar_su_codigo`
7	Encapsulamiento de eventos y comandos	`events/comandos.py`, `events/eventos.py`	`uv run pytest tests/test_eventos_y_comandos.py -k inmutable`
8	Herencia avanzada	`DocumentoPDF` → `DocumentoLocal` → `Documento`	`uv run pytest tests/test_documentos.py -k hereda`
9	Herencia simple, múltiple, multinivel, jerárquica	`domain/documentos.py`, `domain/contratistas.py`, `domain/mixins.py`	`demo.py` sección 2 (MRO); `uv run pytest tests/test_contratistas.py -k mro`
10	Composición frente a herencia	`domain/cuenta_cobro.py`, `CuentaCobro`; servicios por constructor	`uv run pytest tests/test_cuenta_cobro.py -k "hereda or composicion"`
11	Liskov	Subclases de `Documento`; repositorios memoria/Mongo	`demo.py` sección 3; `uv run pytest tests/test_documentos.py -k lsp`; `uv run pytest -m integracion tests/test_repositorios.py`
12	Riesgos de jerarquías profundas	Máximo 3 niveles	`uv run pytest tests/test_documentos.py -k niveles`

#	Tema	Dónde está	Cómo demostrarlo en vivo
13	Polimorfismo dinámico y late binding	`EventoDominio.__str__` llama a `describir()`; `Documento.__post_init__` llama a `validar()`	`uv run pytest tests/test_eventos_y_comandos.py -k polimorfico`
14	Sobrescritura y despacho dinámico	`validar()` y `describir()` en cada subclase	`uv run pytest tests/test_documentos.py -k "pdf or ejecucion"`
15	Métodos abstractos e interfaces	`Documento`, `Contratista`, `EventoDominio`, `EventPublisher`, `EventSubscriber`, repositorios	`uv run pytest -k "instanciar or abstracto"`
16	Duck typing	`ManejadorEventos` (Protocol) en `events/publicador.py`	`demo.py` sección 5; `uv run pytest tests/test_bus_eventos.py -k duck`

Componentes adicionales, fuera del temario:

Componente	Dónde está	Cómo demostrarlo
Autenticación y roles	`domain/usuarios.py`, `services/servicio_auth.py`, `api/seguridad.py`	Entrar con los 5 usuarios; `uv run pytest tests/test_auth.py`
Matriz rol por endpoint	`tests/test_api_completa.py::test_permisos_por_rol`	22 endpoints por 5 roles
GridFS e integridad	`domain/archivos.py`, `RepositorioArchivosGridFS`	Subir y descargar en la plataforma; `uv run pytest tests/test_api_completa.py -k documentos`
Auditoría persistente	`events/auditoria.py`, `AuditoriaMongo`	Pantalla Auditoría
Dashboard con agregaciones	`RepositorioCuentasMongo.resumen_por_estado` (`$group`)	Pantalla Resumen
Frontend e2e	`frontend/e2e/flujo-cuenta.spec.ts`	`cd frontend && npm run e2e`

5. Cómo demostrar en vivo

Sin red, en 2 minutos.

```
uv sync
uv run python demo.py
uv run pytest -q
```

`demo.py` muestra en orden: Documento abstracto (TypeError), MRO de `DocumentoPDF` y `PersonaJuridica`, documentos mixtos en un `for` (LSP), el flujo completo con auditoría y notificador, el `ContadorPato` (duck typing) y una transición inválida rechazada.

En la plataforma.

1. Entrar como `ana.rodriguez@entidad.gov.co` / `Demo2025*` y mostrar que el menú sale de los permisos de `/auth/me`.
2. Radicar contra `CONT-2025-004` con fecha entre 2025-03-01 y 2026-02-28.
3. Subir un PDF: el detalle muestra `application/pdf`, el tamaño y el tipo detectado.
4. Enviar a revisión, aprobar y registrar el pago; el historial se dibuja como línea de tiempo.
5. Abrir Auditoría: aparecen `CuentaRadicada`, `DocumentoAdjuntado`, `EstadoCuentaCambiado` y `CuentaPagada` (polimorfismo de eventos).
6. Salir y entrar como `maria.cifuentes@entidad.gov.co` (Revisor): no aparece Usuarios ni Auditoría y no ve el botón de pago.
7. Entrar como `pedro.sarmiento@entidad.gov.co`: usuario inactivo (403). Como `luz.paredes@entidad.gov.co`: cambio obligatorio de contraseña.

Contra Atlas. `uv run pytest -m integracion` corre la misma batería de repositorios contra MongoDB y GridFS reales, en una base temporal que se borra.

6. Riesgos conocidos

Riesgo	Impacto	Mitigación o siguiente paso
Render gratuito se duerme tras 15 minutos	La primera petición tarda hasta un minuto	El frontend muestra "Iniciando el servidor"; un plan pago lo elimina
Tope del contrato sin transacción	Dos radicaciones simultáneas podrían superar el valor total	Transacciones de MongoDB o una actualización condicional
Bus de eventos síncrono y en memoria	Un evento no se reintenta si el proceso cae justo después de guardar	Patrón *outbox* o un broker (RabbitMQ, Kafka) detrás de `EventPublisher`
Atlas abierto a `0.0.0.0/0`	Superficie mayor; la protección es la credencial	Lista de IP de salida de Render (plan pago) o *private endpoint*
JWT sin revocación	Un token robado sirve hasta que vence (60 min)	Lista de revocación o tokens de vida corta con refresco
Fechas del SQL interpretadas como UTC	Los datos de ejemplo pueden verse con 5 horas de diferencia	Normalizar a hora de Colombia al importar
Notificador sin canal real	Solo guarda el mensaje en memoria	Suscriptor de correo detrás del mismo bus

7. Preguntas probables de sustentación

1. **¿Por qué `Documento` es abstracta?** Porque "un documento" en general no existe: siempre es local, PDF o URL. Al ser ABC con `validar` y `describir` abstractos, Python impide instanciarla y obliga a cada hija a implementarlos.
2. **¿Qué es el MRO y dónde se usa?** El orden en que Python busca un método en la jerarquía. Lo usa la herencia múltiple de `Contratista(AuditableMixin, ABC)` y el bus, que recorre `type(evento).__mro__` para entregar a quien se suscribió a una clase madre.
3. **¿Por qué `CuentaCobro` no hereda de `Documento`?** Porque no *es* un documento: *tiene* documentos. Heredar la obligaría a tener ruta y tipo MIME.
4. **¿`DocumentoPDF` viola LSP al exigir `application/pdf`?** No. La exigencia está en la creación. Ya creado, se comporta como cualquier documento y no lanza errores nuevos en `describir` ni `a_dict`.
5. **¿Cómo se evita una jerarquía profunda?** Máximo 3 niveles, verificado por un test. Lo demás se resuelve con composición o un mixin.
6. **¿Diferencia entre comando y evento?** El comando es una intención que puede rechazarse (`RadicarCuenta`); el evento es un hecho ya ocurrido (`CuentaRadicada`).
7. **¿Por qué los eventos son inmutables?** Para que todos los suscriptores vean exactamente el mismo hecho y nadie lo altere después de publicado.
8. **¿Por qué se publica después de guardar?** Para no anunciar algo que no quedó guardado. Si `guardar` falla, no hay evento; lo prueba `test_no_publica_si_guardar_falla`.
9. **¿Qué pasa si un suscriptor falla?** El bus captura la excepción, la registra con `logging` y entrega a los demás.
10. **¿Cómo recibe la auditoría eventos nuevos sin tocar su código?** Está suscrita a `EventoDominio` y el bus recorre el MRO; cualquier subclase nueva le llega.
11. **¿Dónde hay duck typing?** El bus tipa los suscriptores con el `Protocol` `ManejadorEventos`; `ContadorPato` no hereda de nada y funciona.
12. **¿Por qué MongoDB y no el diseño SQL?** Era requisito del curso usar NoSQL. Una cuenta se lee siempre con sus soportes e historial, y embebidos se traen en una sola lectura.
13. **¿Qué se embebe y qué se referencia?** Se embebe lo que solo existe dentro del dueño (permisos en roles; documentos e historial en cuentas). Se referencia por `_id` lo que tiene vida propia (contrato, contratista, usuario).
14. **¿Qué pasó con los `CHECK` del SQL?** Pasaron al dominio: `ValorMonetario` (valor mayor que cero), `Contrato` (fechas) y `DocumentoLocal.validar` (metadatos).
15. **¿Y los `UNIQUE`?** Son índices únicos en `repositories/indices.py`, incluido el compuesto `(id_contrato, numero_cuenta)`.
16. **¿Por qué dos implementaciones de repositorio?** Para correr las pruebas sin red y demostrar LSP: la misma batería pasa en memoria y en Mongo.
17. **¿Por qué no un repositorio genérico?** Cada agregado tiene consultas distintas; un genérico tendría métodos sin uso o filtros sin tipo.
18. **¿Cómo funciona la autenticación?** Login con bcrypt; el backend emite un JWT firmado con `JWT_SECRET`. El BFF lo guarda en una cookie httpOnly y lo reenvía como `Bearer` en cada petición.
19. **¿Diferencia entre 401 y 403?** 401: no se sabe quién es (sin token o token inválido). 403: se sabe quién es pero no tiene permiso, o está inactivo o bloqueado.

20. **¿Cómo se prueba el bloqueo de 15 minutos sin esperar?** `ServicioAuth` recibe un `reloj` por constructor; el test lo adelanta.
21. **¿Por qué el tipo de archivo se valida por contenido?** Para que renombrar un `.exe` a `.pdf` no lo haga pasar: se leen los primeros bytes (`%PDF-` , PNG, JPEG).
22. **¿Qué garantiza la integridad de un adjunto?** Al subir se calcula su sha256; al descargar se recalcula leyendo GridFS y, si no coincide, se responde 409.
23. **¿Qué pasa si falla guardar la cuenta después de subir el archivo?** El servicio borra el archivo de GridFS para no dejar huérfanos.
24. **¿Cómo se evita superar el valor del contrato?** El servicio suma las cuentas no anuladas ni rechazadas (agregación `$group`) y compara con `valor_total` antes de radicar o editar.
25. **¿Qué cambiaría para producción real?** Transacciones en el tope del contrato, un broker de eventos con *outbox*, revocación de tokens, restricción de IP en Atlas y un plan sin suspensión en Render.